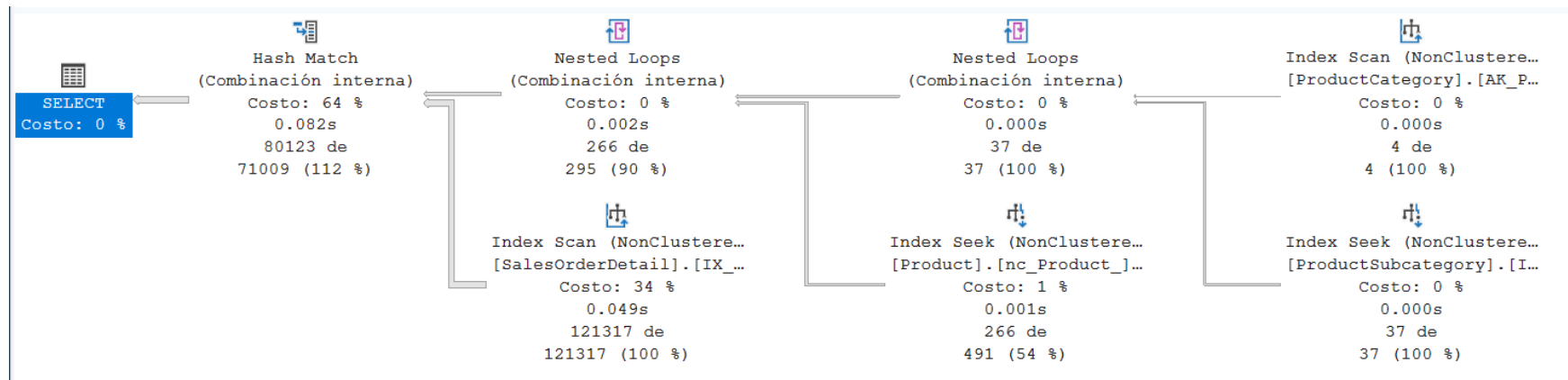


PLANES DE EJECUCIÓN

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
3	16-03-26	1006 ms.	141 ms	SOD= 273 Product=15	55%	Index scan de costo 33% Hash Match costo 63%	Se recomienda realizar un índice en subcategory ID y en categoryID

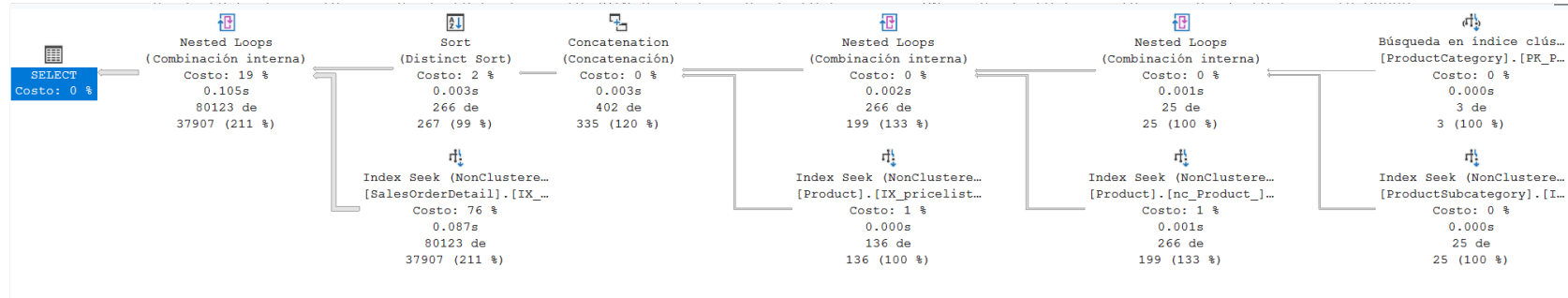


MENSAJES

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, ...
 Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, ...
 Table 'SalesOrderDetail'. Scan count 1, logical reads 273, physical reads 1, ...
 Table 'Product'. Scan count 37, logical reads 76, physical reads 4...
 Table 'ProductSubcategory'. Scan count 4, logical reads 8, physical reads 1, ...

SQL Server Execution Times:
 CPU time = 94 ms, elapsed time = 1012 ms.
 SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 0 ms

Re-escribiendo la consulta:



Mensajes:

Table 'SalesOrderDetail'. Scan count 266, logical reads 908, physical reads 138, page server reads 0, read-ahead reads 50, ...

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...

Table 'Product'. Scan count 26, logical reads 56, physical reads 5, page server reads 0, read-ahead reads 2, ...

Table 'ProductSubcategory'. Scan count 3, logical reads 6, physical reads 1, page server reads 0, ...

Table 'ProductCategory'. Scan count 3, logical reads 6, physical reads 1, page server reads 0, ...

SQL Server Execution Times:

CPU time = 78 ms, elapsed time = 783 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 1 ms.

CONSULTAS:

<pre>SELECT sod.SalesOrderID, p.ProductID, p.Name FROM Sales.SalesOrderDetail sod JOIN Production.Product p ON sod.ProductID = p.ProductID JOIN Production.ProductSubcategory ps ON p.ProductSubcategoryID = ps.ProductSubcategoryID JOIN Production.ProductCategory pc ON ps.ProductCategoryID = pc.ProductCategoryID WHERE pc.ProductCategoryID IN (1,2,3)-- >=1 and pc.ProductCategoryID <=3 -- IN (1,2,3) OR p.ListPrice > 500;</pre>	<pre>with ProductsIds as(select p.ProductId, p.Name from Production.Product p JOIN Production.ProductSubcategory ps ON p.ProductSubcategoryID = ps.ProductSubcategoryID JOIN Production.ProductCategory pc ON ps.ProductCategoryID = pc.ProductCategoryID WHERE pc.ProductCategoryID IN (1, 2, 3) UNION select p.ProductId, p.Name from Production.Product p WHERE p.ListPrice > 500) SELECT sod.SalesOrderID, pi.ProductID, pi.Name FROM Sales.SalesOrderDetail sod inner join ProductsIds pi on sod.ProductID = pi.ProductId ;</pre>
--	---

Creación de índices:

```
CREATE NONCLUSTERED INDEX ix_CSubctegoryid
ON production.productSubcategory (productcategoryid)
include (productsubcategoryid);
GO

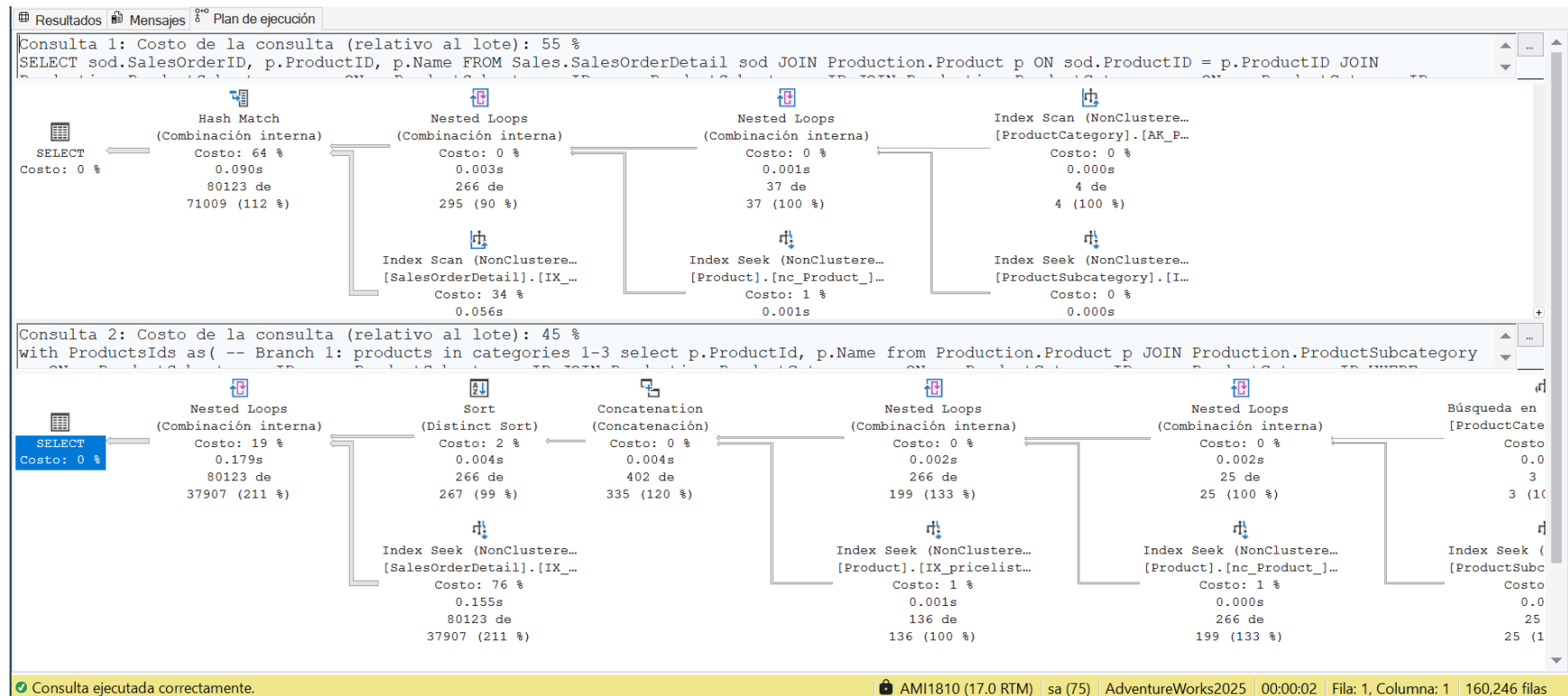
CREATE NONCLUSTERED INDEX ix_pricelist
ON production.product (listprice)
include (productid, name);

CREATE NONCLUSTERED INDEX nc1_SalesOLrderDetail_ProductID
ON Sales.SalesOrderDetail(ProductID)

-- indice Product con columnas necesarias
CREATE NONCLUSTERED INDEX nc_Product
ON Production.Product (ProductSubcategoryID)
INCLUDE (Name, listprice);

-- indice subcategory relacionar subcat y cat
CREATE NONCLUSTERED INDEX nc_Subcategory_CategoryID
ON Production.ProductSubcategory (ProductCategoryID)
INCLUDE (ProductSubcategoryID);
```

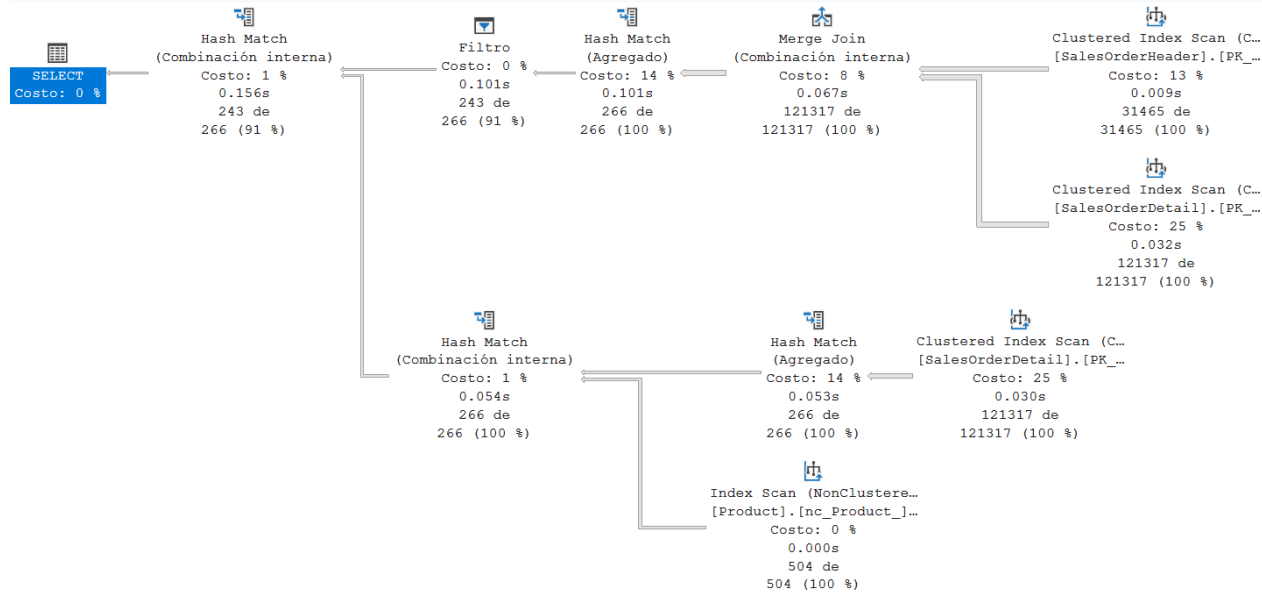
Comparación de los planes de ejecución:



Conclusion:

Aunque casi se triplicaron las lecturas lógicas de la tabla SalesOrderDetails, mejoro en tiempo de ejecución y en el costo total de la consulta. Como primera mejora, se pudo eliminar el Hash Match por un Nester Loop ya que se están obteniendo solo los valores filtrados para la visualización. Además, por la creación de los índices, podemos ver que se están utilizando index seek en lugar de index scan lo cual hace que lea directamente las filas y reduce las operaciones de entrada y salida.

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
8	20- 03-26	365 ms	141 ms	Sod = 2482 Soh=686 P=6	62%	Clustered index scan Hash Match	Realizar una reescritura de la consulta e implementar índices en soh y sod



Mensajes:

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
Table 'Product'. Scan count 1, logical reads 6, physical reads 0, page server reads 0, read-ahead reads 0, ...
Table 'SalesOrderDetail'. Scan count 2, logical reads 2482, physical reads 0, page server reads 0, ...
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 0, page server reads 0, ...

SQL Server Execution Times:

CPU time = 141 ms, elapsed time = 365 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 0 ms.

Re-escritura de consulta:

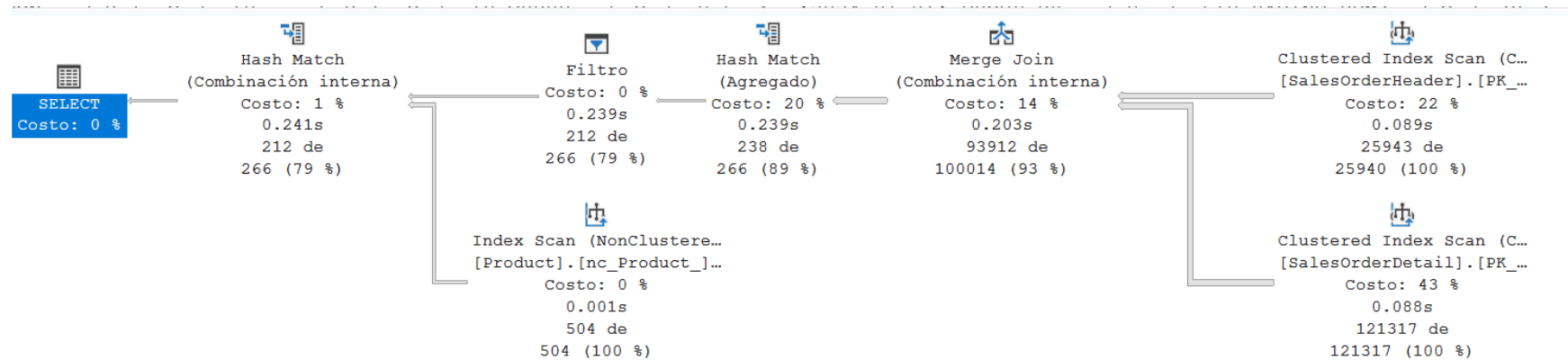


Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
 Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
 Table 'Product'. Scan count 1, logical reads 6, physical reads 1, page server reads 0, read-ahead reads 4, ...
 Table 'SalesOrderDetail'. Scan count 1, logical reads 1241, physical reads 166, page server reads 0, read-ahead reads 1237, ...
 Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 378, page server reads 0, read-ahead reads 682, ...

SQL Server Execution Times:

CPU time = 203 ms, elapsed time = 371 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 0 ms.

Con creación de índices:

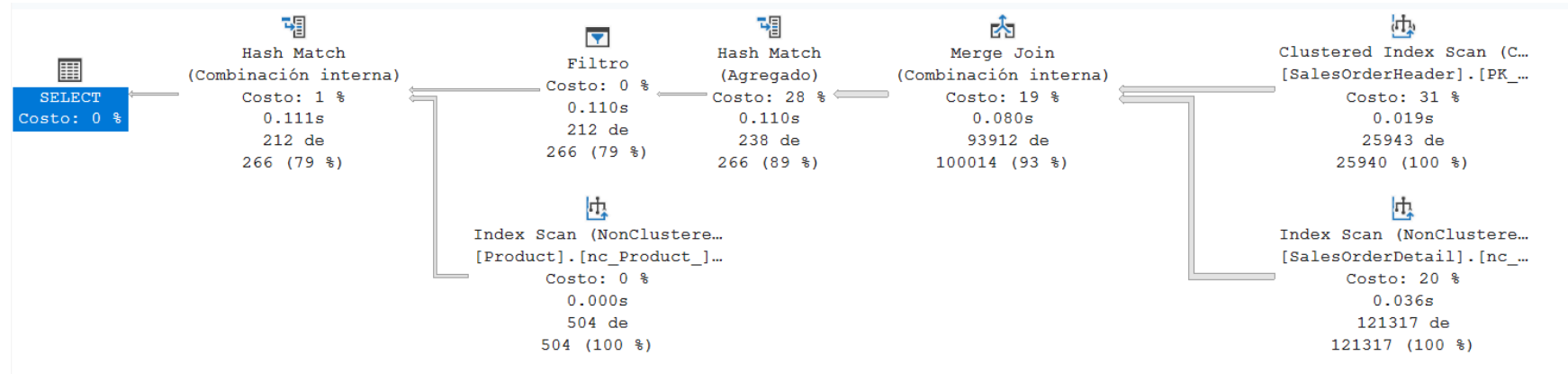


Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
Table 'Product'. Scan count 1, logical reads 6, physical reads 0, page server reads 0, read-ahead reads 0, ...
Table 'SalesOrderDetail'. Scan count 1, logical reads 303, physical reads 1, page server reads 0, read-ahead reads 249, ...
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3, page server reads 0, read-ahead reads 561, ...

SQL Server Execution Times:

CPU time = 94 ms, elapsed time = 344 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 0 ms.

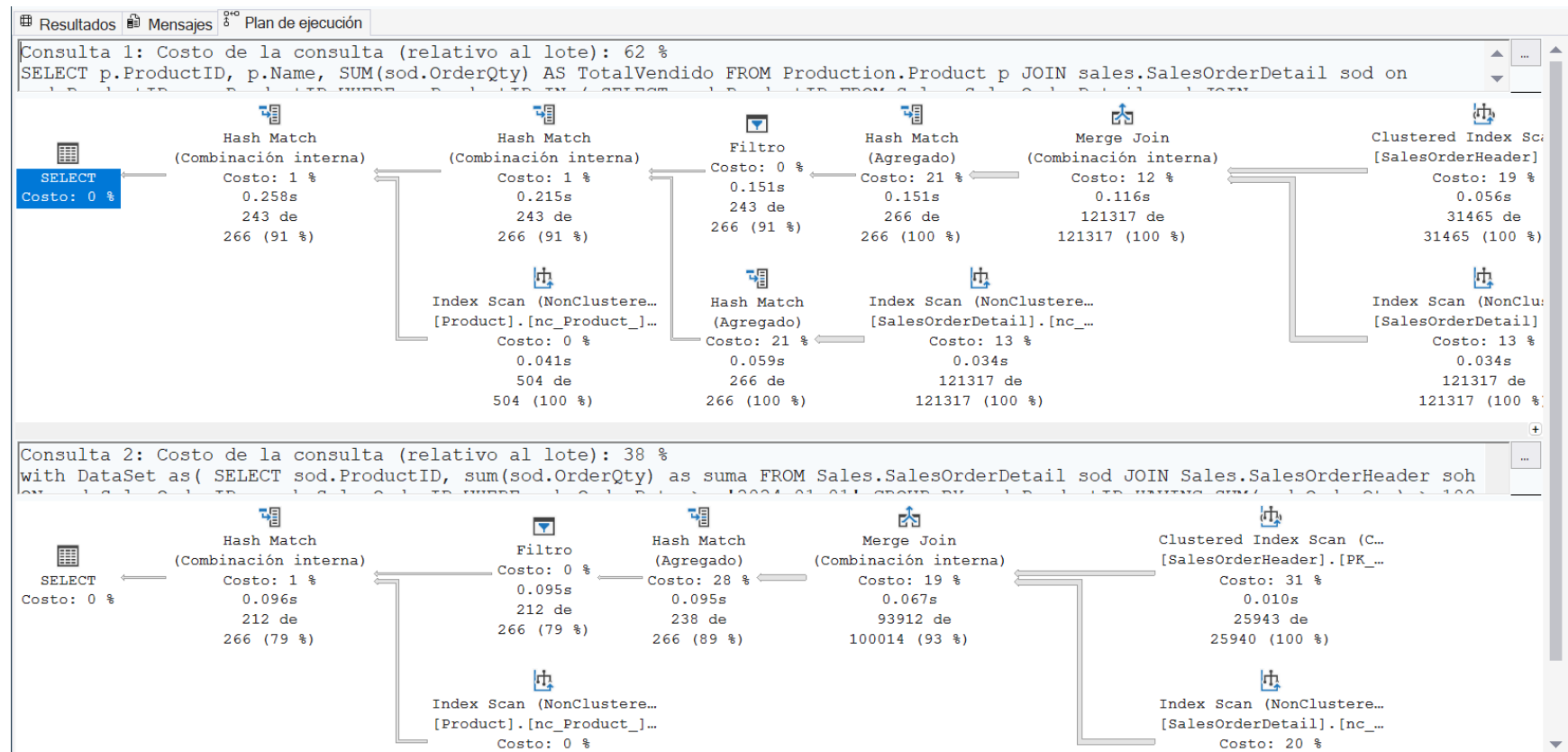
Consultas:

Original	Re-escritura
<pre> SELECT p.ProductID, p.Name, SUM(sod.OrderQty) AS TotalVendido FROM Production.Product p JOIN sales.SalesOrderDetail sod on sod.ProductID = p.ProductID WHERE p.ProductID IN (SELECT sod.ProductID FROM Sales.SalesOrderDetail sod JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID WHERE soh.OrderDate >= '2014- 01-01' GROUP BY sod.ProductID HAVING SUM(sod.OrderQty) > 100) GROUP BY p.ProductID, p.Name; </pre>	<pre> with DataSet as(SELECT sod.ProductID, sum(sod.OrderQty) as suma FROM Sales.SalesOrderDetail sod JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID WHERE soh.OrderDate >= '2024- 01-01' GROUP BY sod.ProductID HAVING SUM(sod.OrderQty) > 100) SELECT p.ProductID, p.Name, ds.suma AS TotalVendido FROM Production.Product p inner join DataSet ds on p.ProductID = ds.ProductID </pre>

Índices

<pre> CREATE NONCLUSTERED INDEX nc_SalesOrderHeader_OrderDate ON Sales.SalesOrderHeader (OrderDate); CREATE NONCLUSTERED INDEX nc_SalesOrderDetail_OrderID_Cover ON Sales.SalesOrderDetail (SalesOrderID) INCLUDE (ProductID, OrderQty); </pre>
--

Comparación de plan de ejecución:



Conclusión

Al implementar una CTE se puede ver notoriamente el cambio aun antes de implementar los índices en donde no se hace una doble consulta a la tabla SalesOrderDetail. Al crear los índices con las columnas que ocuparemos, evitamos realizar demasiadas lecturas lógicas a la tabla SOD.